Computational Introduction to Robotics Project 2: State Localization

Thanadetch (Detch) Mateedunsatits dmateedunsatits@olin.edu

Ophelia Lonzo olonzo@olin.edu

October 20, 2026

1 Project Overview

The goal of this second project is to learn how to find the robot's location and work with localization algorithms like the particle filter. Here is the link to the assignment page for a more detailed description of the assignment. In this project, our robot is given a map of the first floor of the MAC, but does not know where on the map it is located. We have to create a particle filter so that the robot can figure out where it is. We approached this by creating a particle filter that creates 1000 pose guesses, then uses odometry and lidar scans to weigh them against each other. We were given a starter file pf.py, that handled the ROS components, i.e., subscriptions, scans, and publishing, for the algorithm's 5 steps. We filled out these steps with our own logic. Some key design choices we made were only using a subset of the beams per scan, using a circular mean for heading, and making new particle objects with noise when resampling particles and even how to initialise the particles. Ultimately, we created an adaptive sampling particle filter that does random initialization in the beginning. One big takeaway from this project is the importance of unit testing our math outside of the larger ROS tests to simply debug. Another takeaway was there's many ways to achieve the same goals with different algorithms and logical paths to get there, such as our decision to change our sampling condition.

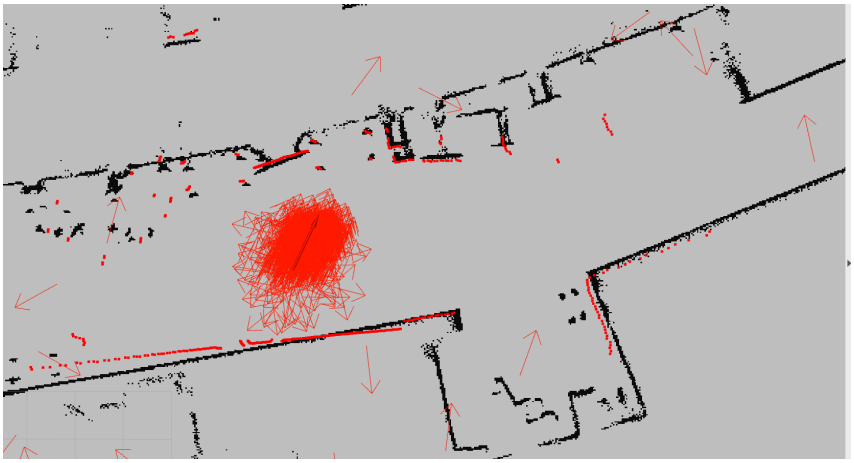


Figure 1: Our particle filter running on the macfirst_floor_take_1 bag in rviz. Most particles (red arrows) have converged into a single cluster at the robot's estimated pose, and the laser scan (red points) lines up with the walls of the map (black). The scattered arrows elsewhere on the map are randomly injected particles.

2 Approach to state estimation

We approached the particle filter by first initializing, applying the motion update, updating using the laser, pose estimating, and then resampling when n_eff is low. This process is run every time the robot moves a set amount. Our threshold for this is 0.2 meters of movement and pi/6 rads of rotation. We create 1000 particles per process, each particle object having an x,y, theta, and weight component.

We generate particles around a valid bounding box and then take a uniform distribution of x, y, θ values to generate initial particles. We then use motion updates that utilize gaussian noise in order to account for errors in odometry and generate more variable data. We are converting from odom to map during this process too.

When generating weights, we pretend that the robot is actually at each particle pose. We then compare the laser scans with the function get_closest_obstacle_distance, and then score each particle by how close the scan is to the helper function. We do this calculation in log form to prevent underflows to zero. We also chose to use only 40 evenly spaced beams for speed, as using all of the beams would make the calculations take a long time. We then normalize the weights so that they sum to 1.

To calculate the estimated position, we take the weighted mean of the particles. We average the heading as a circular mean, because, for example, the plain average of 179 and -179 is 0, which is in the opposite direction. To take the circular mean, we turn each heading into a unit vector ($\cos\theta$, $\sin\theta$), take the weighted sum of these vectors, and use atan2 to find the angle of the result. For 179 and -179, both vectors point almost straight left, so their sum also points left, giving a mean of 180.

After creating a pose estimate, we resample using the draw_random_sample helper function to create particles with probability proportional to their weight, create particles with respect to robot_pose, and create entirely random uniform conditions. Resampling only happens when the n_eff value drops below 0.4, meaning that less than 40% of the particles are actually contributing good data. We then create new Particle objects with Gaussian noise to spread out the particles slightly.

We created a couple of unit tests to verify the math in our filter. The first unit test tests if the weight of normalized particles actually adds up to one. The second unit test tests a guard that we made to make sure that, in a case where all weights are 0, we do not try to divide by 0. The third unit test makes sure that after resampling works as we intended.

3 Algorithm Overview

A particle represents some value of x, y, θ, and weights. We used 1000 of these particles within our algorithm. Our algorithm has a few conditions for updating such as the Neato moving 0.2 meters or turning pi/6 rads.

Initialization

Our filter does not begin with a guess of the robot's pose. Instead, these particles are initialized entirely randomly through getting a uniform distribution of x and y values from a bounded grid. These bounded grids can be found from the map the file is run on. A more sophisticated method and an extension of this project would be to only initialize these particles where the map is defined as movable or valid; however, we chose to take the simple naive method of simply taking a uniform distribution over all x and y values within that grid given we are sending 1000 particles. Within our code is also the option to initialise around a given 2D pose estimate, which would be an alternative way of initializing these data.

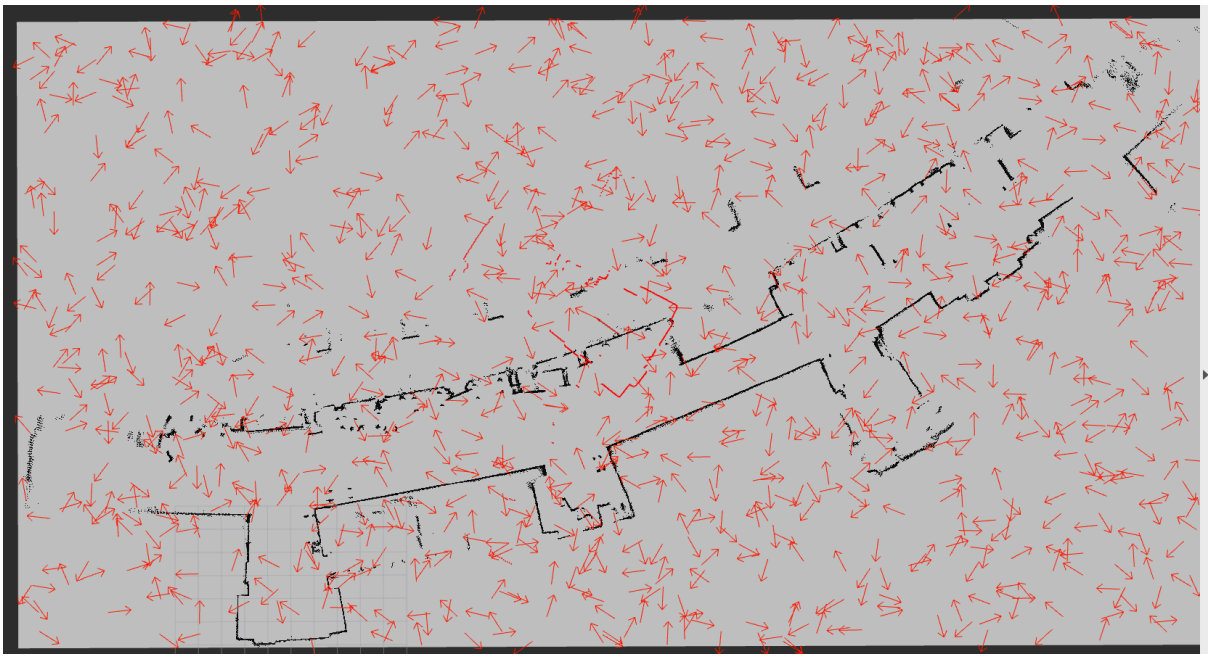


Figure 2 demonstrates global initialization where 1000 particles are spread uniformly over the bounding box of the map with random headings. Many particles start outside the building, since we did not check whether a particle is within the border of our bag files.

Update particle positions

We read the value of the Neato's odometry and sensor updates in order to make decisions about our laser scans while applying a layer of gaussian uncertainty. Below is a figure demonstrating we implemented the change in odometry to map frames. Additionally, with all of this we added a gaussian noise to both the x,y,θ that is modelled with a gaussian that has varying standard deviation equal to 10% of the absolute change in the x, y, or θ plus some minimum 1 centimeter of noise or 1 degree of noise.

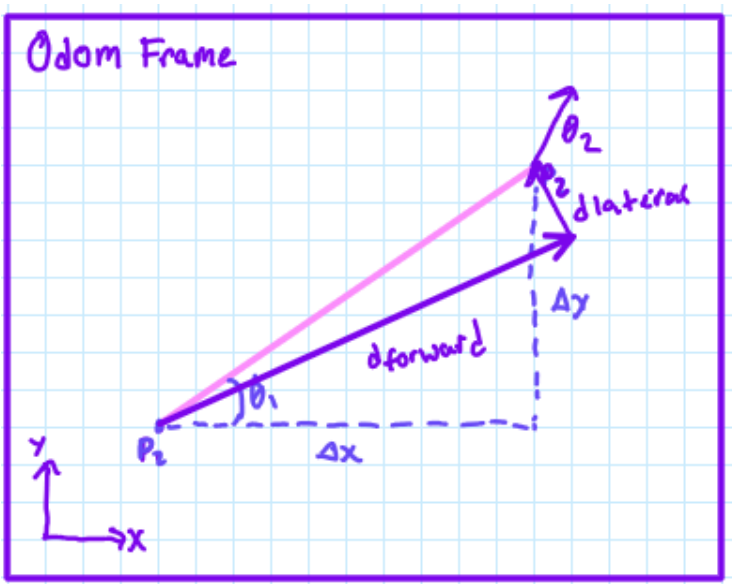


Figure 3: The change in odometry (Δx, Δy) between two updates is split into d_forward, along the robot's previous heading θ_1, and d_lateral, perpendicular to it. These are then applied relative to each particle's own heading.

d_forward = $\Delta x \cos \theta_1 + \Delta y \sin \theta_1$,
d_lateral = $-\Delta x \sin \theta_1 + \Delta y \cos \theta_1$,
x becomes x + d_forward cos θ - d_lateral sin θ
y becomes y + d_forward sin θ + d_lateral cos θ
θ becomes θ + Δθ

Updating robot pose

The robot pose is defined as the weighted mean of the particles with the following equations.

$$\hat{x} = \sum_{i=1}^{n} w_i x_i \qquad \hat{y} = \sum_{i=1}^{n} w_i y_i \qquad \hat{\Theta} = atan2(\sum_{i=1}^{n} w_i sin(\Theta_i), \sum_{i=1}^{n} w_i cos(\Theta_i))$$

Resampling and Adaptive resampling

We resample using a method called adaptive resampling where when a metric we used called the effective sample size is less than 0.4 and this helps make sure we do not have issues with degenerative particles, an issue where a few particles dominate the weights of the function. The effective sample size is defined as the size of a simple random sample of independent observations that would provide the same level of statistical precision as a weighted dataset. We want these values to be closer to 1 which would represent all equally contributing particles instead of 0 which would represent only a few particles contributing to the weights.

Whenever resampling is triggered, we do a majority weighted sampling where it is 80% based on the particles with some small jitter like 2 centimeters and 0.02 rad, both of which are gaussian noise. Additionally, we injected 10% of particles around the local robot pose with random angles and 30 centimetres of standard deviation. Lastly, we kept 10% of the particles as entirely random to give the robot a good chance to recover in the case of a bad-guess.

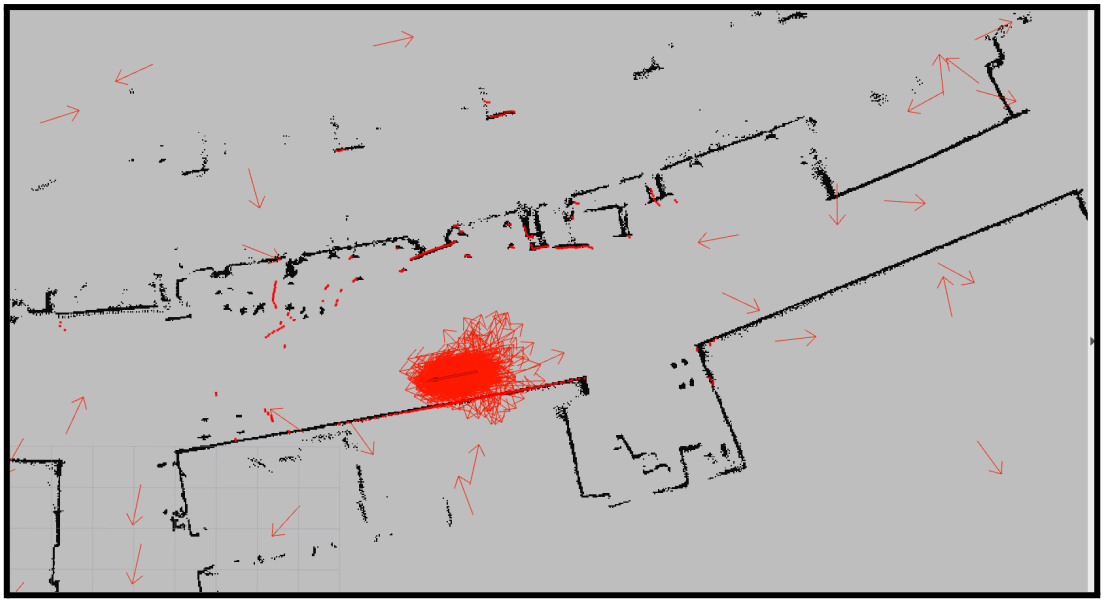


Figure 4: The particle cloud after several resampling steps. The weighted particles form a tight cluster along the corridor, and the laser scan lines up with the walls. The arrows spread across the map are the 10% of particles injected at random to help the filter recover from a bad guess.

Laser Scans

For each particle, a subset of beams is projected into the map and the distance from each hit to the nearest obstacle is treated as error. Weights are computed in log space for numerical stability. Each beam's distance-to-nearest-obstacle **d** is squared and summed, and the weight is $w_prev \cdot \exp(-tempering \cdot \Sigma d^2 / (2 \cdot \sigma^2))$, where tempering = 0.5, sigma = 0.4. Because we do not resample after every update, the weights do not reset between steps, so we multiply each particle's previous weight by the new scan likelihood (which becomes an addition in log space). This way, data from earlier scans is not lost when resampling is skipped.

4 Implementation Design

One key design decision we made was to incorporate adaptive resampling; this means that the program will only resample the particles when less than 40% of the particles are ineffective. Another design decision was to make the robot not make an initial pose guess but instead generate random particles around the bounded area. Within this design decision lies another decision on whether it made sense to have a logic condition check on whether a degenerated particle lies within the bounded area, for the sake of simplicity, we just naively took a uniform sample even if many particles are wasted in the beginning.

We also added a small amount of noise in each motion update, so if the robot is stationary, the cloud of particles does not spread out. We used log-space weights to prevent very small weights from compounding into something that will be rounded to 0. We chose to only use 40 beams of the lidar scan for speed.

5 Unit Test Design and Integration

For our unit tests, we created a particle filter using the __new__ flag so ROS does not start and we can set only the attributes we need. We also used MagicMock and mocked angle_diff as plain subtraction to avoid having to actually run ROS. Our first 3 unit tests test the normalization and resampling math. The first test, test_normalize_sums_to_one, tests that after weights are normalized, they sum to one, as the name implies. The second test tests that a guard we created in our code works. This guard sets all weights to an identical value if all weights are 0, to prevent a crash due to division by 0. Our third test tests that resampling works as intended, i.e., that it prioritises larger weights and that all particles are unique. We then test that the odometry update moves particles in the direction they are facing, not in the direction the robot is facing. We then test that if that robot only rotates in place, particles will do the same and not move. We finally check that the reference pose is also updated in the odometry update. To run these tests, run **PYTHONNOUSERSITE=1 python3 -m pytest <test_pf.py > -v** in the robot_localization directory.

6 Experimental Design

We tested our design through using unit tests during the debugging phase and using bag files as well as print statements and loggers for n_eff, later on we moved all our print statements into the logger. With just visual inspection, one of our first few runs had issues, unfortunately, we did not record bag files but we had the sneaking suspicion that it wasn't sampling too well. This led us to learn about the n_eff metric and we implemented it. This made our data a lot better and our runs more stable over time.

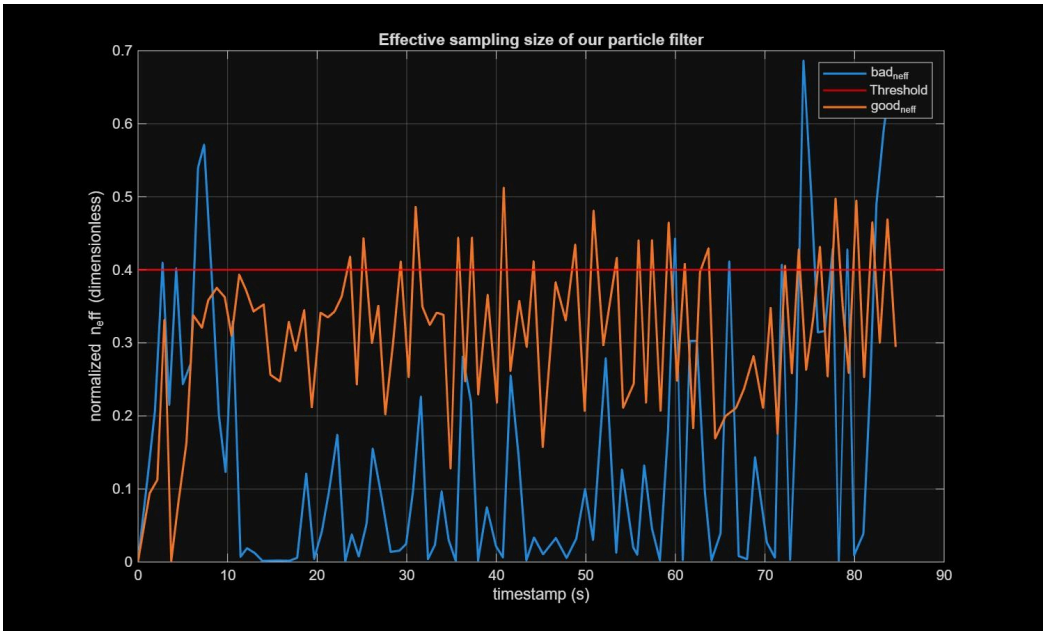


Figure 5: The graph shows the values of n_eff over time compared to a graph of what the threshold is. A higher n_eff value is better, a lower n_eff value means a few particles are dominating the weights.

Our project also has multiple bag files demonstrating the behavior of our particle filter. Since it does not have an initial guess, it often takes time to converge upon a position. We've linked two specific demonstrations of our filter. One where it got a lucky guess and was close to the original position (found in pf_global_test2 in bags) and another one where it was very unlucky but eventually located itself anyways (found in pf_ global_test4).

pf_global_test2 Good Data
https://youtu.be/KEfLwFJEzf0?si=16PdJTaSpphj_XW4

pf_ global_test4 Recovery
https://youtu.be/vxPtXP_Gg_U

7 Challenges, Next Steps, Improvements

We caught a lot of bugs through unit testing, one big one being dividing by a list instead of an element of the list. We also had issues with numpy when unit testing, and had to run the test with the PYTHONNOUSERSITE=1 flag to get the tests to go through. We also had some issues with angle_diff in our unit tests, and had to make a fake function with plain subtraction to use instead. Another big challenge we faced was figuring out how fast the particles should converge, and figuring out what constants should be to achieve that. This problem took a lot of testing and iteration to figure out.

We also had issues with the particle filter having degenerate particles in one of our earlier implementations so we changed our sampling method to be an adaptive sampling method. Afterward, we also introduced more noise, randomness, and other behaviors in order to make the project more robust. We also had an issue where our original versions had too little variation and depended too much on a good first robot pose. It would converge almost instantly within a few timesteps. We shifted our methods entirely into creating a bounded box and then creating random uniform particles within that boundary from there. This slowed our convergence process more and gave the robot more chance to properly guess its position. This design should make the robot more resistant towards bad first guesses.

In terms of the next step, a quick improvement would be to bound all our initial guess to the borders of the map which would greatly increase the density of each guess and help the process converge faster and potentially more accurately.

8 Learning Objectives and Final Takeaways

Ophelia -
My learning goal for this project was to get more comfortable with testing code in ROS. Writing unit tests for this project really helped with this, as I had to figure out how to run them without accessing ROS. I also wanted to learn more about how to choose constants like how much noise to add and how many beams we needed. I learned a lot about how to do that effectively while testing this project.

Detch -
My goal was to learn about the mathematics and understanding behind how robots localize and where they are on a map. I'm glad to have learned it through the particle filter. I also enjoyed learning more about the concept of what is good/bad sampling and implementing the adaptive sampling filter of the project.

9 How To Run

To run the particle, first get the code and build it:

```
cd ~/ros2_ws/src
git clone [your repository URL] robot_localization
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

Use three terminals:

Terminal 1: the particle filter and the map server

```
ros2 launch robot_localization test_pf.py

map_yaml:=/home/<your-username>/ros2_ws/src/robot_localization/maps/mac_
1st_floor_final.yaml
```

Terminal 2: rviz

```
rviz2 -d ~/ros2_ws/src/robot_localization/rviz/turtlebot_bag_files.rviz
```

Terminal 3: the bag file

```
ros2 bag play
~/ros2_ws/src/robot_localization/bags/macfirst_floor_take_1 --clock
--start-paused
```

1. In Terminal 3, press the spacebar to start the bag. Let it run for 2 to 3 seconds.
2. Press the spacebar again to pause it.
3. Press the spacebar in Terminal 3 to resume the bag.

*It is also possible to play other bag files by passing it into terminal 3 and replacing the name of the file being played from **macfirst_floor_take_**1 to the bag file name (see bag file folders).

10 Acknowledgements

Claude Opus 5.5 (medium) was used to create the file to quickly convert rosbags into .mp4 files and the prompt used was “try to turn the rosbag files for the globalstart[number] into a video format like .mp4.” These video files are under the → Videos section of the repository and referenced in the experimental design aspect. It was used on October 21, 2:16 A.M.